Business Intelligence Assistant for Small Enterprises

By Carson Lane

CMPT 496

Working report draft

Contents

1. Introduction

- **1.** Literature Review
- **2.** Goals of the Project

2. Methodology

- **1.** Overview
- **2.** Application
  - **1.** Software
  - **2.** Software Design

3. Results

- **1.** Testing
- **2.** Conclusions
- **3.** Future Work

4. References

5. Appendices

Abstract

Small enterprises may collect useful business data without having the specialized tools or expertise needed to prepare and interpret it. This project develops a lightweight Business Intelligence Assistant that guides users through uploading sales data, reviewing validation findings, cleaning supported issues, and exploring sales metrics and visualizations. The prototype is implemented in Python with Streamlit and pandas. Its workflow includes validation, data previews, automated cleaning, a sales dashboard, and generated business insights.

The current application supports sales CSV files containing `date`, `amount`, `category`, and `item` columns. It does not currently support arbitrary business datasets, database integration, or forecasting. Development testing has used clean, messy, and structurally invalid sales fixtures. This report describes the system design, implementation, testing, limitations, and possible future development.

Keywords: business intelligence, small enterprises, data quality, data cleaning, sales analytics, self-service analytics

1. Introduction

Business Intelligence (BI) and Business Intelligence and Analytics (BI&A) support organizational decision-making by helping users collect, transform, analyze, and visualize business data [1]. Small and medium-sized enterprises (SMEs) may benefit from these capabilities, but can face cost, infrastructure, and expertise barriers when adopting traditional BI platforms [2], [3].

Many organizations already store operational information in CSV files. However, missing values, inconsistent formatting, duplicate records, and invalid dates can make direct analysis difficult and may reduce confidence in results. A focused application that explains validation findings and automates a limited set of cleaning steps can help make basic analysis more approachable.

This project implements a guided BI workflow for sales CSV files. Users upload a file, inspect its structure and validation results, run supported cleaning, and then explore sales summaries and generated insights. The application is intended as a prototype and is not a general-purpose BI platform.

1.1 Literature Review

BI&A has developed alongside the growing availability of business data and the need to support evidence-based decisions [1]. Research on SMEs discusses potential benefits including improved operational performance, planning, and competitiveness. Llave's review surveys BI&A components used in SMEs [2], while Tsiu et al. review data-mining and BI applications and their potential competitive advantages [3].

Adoption remains challenging for smaller organizations. Limited resources, technical expertise, infrastructure, and organizational readiness may constrain implementation [2]–[5]. These issues motivate self-service BI (SSBI), which aims to let non-specialist users perform analysis without

depending on a dedicated BI team. Research on SSBI highlights usability, accessibility, visualization, and training as important considerations [8], [9].

Data quality is also essential for useful analysis. Dirty data can include missing values, duplicate records, invalid entries, contradictions, and inconsistent representations [10]. Data-cleaning research describes validation, standardization, deduplication, and transformation as important preparation tasks [11]. Broader information-quality work emphasizes dimensions such as accuracy, completeness, and consistency [12].

This project applies these concepts in a deliberately narrow and transparent workflow: it checks a sales-specific data contract, reports selected quality issues, cleans supported cases, and calculates descriptive summaries. It demonstrates a working prototype rather than resolving the broader organizational challenges of BI adoption.

> **Citation work required:** References [4] and [5] in the supplied PDF do
>
> not identify authors, publication venues, years, or other publication
>
> details. Verify the intended sources and complete those citations, or remove
>
> them and their in-text citations. Verify the bibliographic details of all
>
> references before submission.

1.2 Goals of the Project

The goal is to implement a lightweight Business Intelligence Assistant that helps users analyze sales data through a guided self-service workflow. The project objectives are to:

1. Accept sales data from CSV uploads.
2. Check uploaded data for required columns and selected quality issues.
3. Show an uploaded-data preview and explain validation findings.
4. Clean supported issues and summarize the changes.
5. Present core sales metrics, category summaries, and date-based

visualizations.

1. Generate human-readable observations from the cleaned data.
2. Test the workflow using representative valid, messy, and invalid sales

files.

The scope is currently limited to sales data with the exact required columns `date`, `amount`, `category`, and `item`. Support for other business-data types and forecasting are future possibilities, not current capabilities.

2. Methodology

2.1 Overview



Figure 1. Implemented sales-data analysis workflow.

The application is a Streamlit web application that processes uploaded CSV data in memory using pandas. Its user-facing workflow has four stages:

1. **Upload:** The user selects a sales CSV.
2. **Validation:** The application checks required columns, reports selected

quality metrics, and displays the dataset shape and a preview. If required columns are missing, cleaning is blocked.

1. **Cleaning:** The application applies supported cleaning operations and

displays a summary and preview of the resulting data.

1. **Analysis:** The application calculates sales metrics and aggregations,

displays charts, and generates business insights.

Uploaded and cleaned data are held in Streamlit session state for the interactive workflow. The current prototype does not store datasets in a database or provide persistent upload storage. The application architecture and data flow are described in the architecture document.

2.2 Application

2.2.1 Software

Python is the primary programming language. pandas is used for CSV data handling, validation, cleaning, transformation, and aggregation. Streamlit provides the web interface, workflow controls, metrics, and charts. The current project uses Streamlit's charting components; Plotly and Scikit-learn-

based forecasting are not part of the implemented application. Git and GitHub are used for version control.

2.2.2 Software Design

The application is organized into a presentation layer and backend modules with separate responsibilities:

- `frontend/streamlit_app_v2.py` implements the upload and staged user

workflow, including validation, cleaning, dashboard, and insights screens.

- `backend/validation.py` checks required columns and reports missing values,

amount-format issues, invalid dates, duplicates, and currency-formatted amounts.

- `backend/cleaning.py` fills supported missing values, converts amount

values, parses dates, removes exact duplicate rows, and normalizes category labels.

- `backend/transformation.py` calculates KPIs and category/date sales

aggregations.

- `backend/insights.py` creates human-readable observations from the

transformed sales data.

The workflow expects the columns `date`, `amount`, `category`, and `item`. Cleaning removes dollar signs and thousands separators from amounts, normalizes category labels, and removes exact duplicate rows. Date strings should use ISO-like formats such as `YYYY-MM-DD`; unparseable dates are not guessed or repaired. Invalid or missing dates are excluded from time-based analysis but remain part of non-date KPI calculations. The exact behavior and limitations are documented in the architecture document.

The application includes a downloadable text report of generated insights. It does not currently offer database integration, arbitrary dataset selection, persistent analysis storage, or predictive forecasting.

3. Results

The implemented prototype provides an end-to-end path from sales CSV upload to descriptive analysis. Users can see validation findings and previews before and after cleaning. The interface summarizes cleaning changes and shows sales KPIs, category/date visualizations, and generated insights. Missing required columns are reported and prevent the cleaning workflow from proceeding.

These results demonstrate the implemented prototype and its tested scenarios; they do not establish usability across a broad user population or measurable business impact. Invalid dates cannot be reliably corrected without more information, and the current descriptive analytics should not be presented as forecasting or causal analysis.

3.1 Testing

The sales fixture collection covers ordinary and larger clean datasets, missing values, duplicate records, inconsistent category labels, currency-formatted amounts, malformed dates, zero-revenue data, and a file that omits a required column. The fixture inventory and manual test procedure are in the testing document.

The current fixture set is summarized below.

Fixture	Test scenario
`sales_sample_10.csv`	Small, ordinary sales file; pipeline smoke-check input.
`sales_clean_100.csv`	Clean sales file with 100 rows.
`sales_clean_1000.csv`	Clean sales file with 1,000 rows.
`sales_messy_11.csv`	Small file with multiple data-quality issues.
`sales_messy_1000.csv`	Larger messy file with missing values, duplicates, inconsistent categories, malformed dates, and formatted amounts.
`sales_returns_100.csv`	Negative-amount/return cases.
`sales_single_category_50.csv`	Single-category aggregation case.
`sales_validation_issues_5.csv`	Small validation and cleaning scenario.
`sales_yearly_trend_365.csv`	Daily sales records spanning a year.
`sales_zero_revenue_12.csv`	Zero-revenue edge case.
`sales_missing_amount_2.csv`	Invalid schema missing the required `amount` column.

Previous development verification exercised the upload, validation, cleaning, and dashboard/insight workflow against ten valid sales fixtures. The structurally invalid fixture, which omits `amount`, was reported as missing a required column and blocked from cleaning. These are development verification results; the report should be updated with the exact final test date, environment, commands, and outputs before submission.

For the final evaluation, document:

- The Python and package versions and the commands used.
- Results for each fixture, including expected and observed behavior.
- Any automated test output and failures.
- Screenshots of the main workflow, numbered and captioned if included.
- The scope and limitations of the fixture-based evaluation.
- User-evaluation participants and results, if a user study is conducted.

The existing `tests/test_pipeline.py` is a print-based smoke check using one sample CSV, not a comprehensive automated test suite.

3.2 Conclusions

The project implements a focused sales-data assistant that combines data validation, supported cleaning, transparent previews, descriptive metrics, visualizations, and generated insights in a guided interface. Testing with valid fixtures and a missing-required-column case demonstrates the core workflow and one structural failure path.

The system is limited to sales CSV files and a defined set of cleaning rules. It does not currently support other dataset types, database-backed storage, or forecasting. Fixture testing verifies selected technical scenarios but does not alone demonstrate usability or improved business decisions.

3.3 Future Work

Potential future work includes evaluating the interface with users, supporting additional explicitly defined business-data schemas, and refining dashboard visualizations. Other possibilities include downloadable cleaned datasets, persistent analysis storage, and forecasting with historical data. Each extension would require a clear data contract and evaluation plan. These features are proposed future work and are not implemented in the current prototype.

4. References

[1] H. Chen, R. H. L. Chiang, and V. C. Storey, “Business Intelligence and Analytics: From Big Data to Big Impact,” *MIS Quarterly*, vol. 36, no. 4, pp. 1165–1188, 2012.

[2] M. R. Llave, “Business Intelligence and Analytics in Small and Medium-Sized Enterprises: A Systematic Literature Review,” *Procedia Computer Science*, vol. 121, pp. 194–205, 2017.

[3] S. V. Tsiu, M. Ngobeni, L. Mathabela, and B. Thango, “Applications and Competitive Advantages of Data Mining and Business Intelligence in SMEs Performance: A Systematic Review,” *Businesses*, vol. 5, no. 2, 2025.

[4] “Business Intelligence Adoption for Small and Medium Enterprises: Conceptual Framework.” **Incomplete citation; publication details required.**

[5] “Systematic Review of Factors Influencing Adoption of Business Intelligence Systems.” **Incomplete citation; publication details required.**

[6] S. Arunatileka, A. Ginige, A. Hol, and R. Lawson, “SMEs Gaining Competitive Advantage Through eCollaboration,” in *Proceedings of ACIS 2005*, Sydney, Australia, 2005.

[7] D. Arnott and M. Gibson, “The Evaluation of Business Intelligence: A Case Study in a Major Financial Institution,” in *Proceedings of ACIS 2005*, Sydney, Australia, 2005.

[8] M. Durra and G. Al-Naymat, “Self-Service Business Intelligence: Meeting User Needs in a Data-Rich World,” in *2023 24th International Arab Conference on Information Technology (ACIT)*, 2023.

[9] C. Lennerholt, J. van Laere, and E. Söderström, “Implementation Challenges of Self-Service Business Intelligence: A Literature Review,” in *Proceedings of the Hawaii International Conference on System Sciences*, 2018.

[10] W. Kim, B.-J. Choi, E.-K. Hong, S.-K. Kim, and D. Lee, “A Taxonomy of Dirty Data,” *Data Mining and Knowledge Discovery*, vol. 7, pp. 81–99, 2003.

[11] E. Rahm and H. H. Do, “Data Cleaning: Problems and Current Approaches,” *IEEE Data Engineering Bulletin*, vol. 23, no. 4, pp. 3–13, 2000.

[12] C. Batini and M. Scannapieco, *Data and Information Quality: Dimensions, Principles and Techniques*. Springer, 2016.

5. Appendices

- **Appendix A – Code:** Identify the final repository and the relevant

frontend and backend modules.

- **Appendix B – Diagrams:** Include the architecture/workflow diagram and

relevant interface figures, with captions and references in the report.

- **Appendix C – Test Data:** Describe the sales fixture scenarios and link to

the repository's sales test data. Avoid reproducing large CSV files in the report.